architecture, where it would fail on a system with a finer-grained set of memory-ordering instructions such as SPARC.

If you need your application to be portable across a range of target systems, it's important to test it on representative instances of those systems. This is why I listed the processor architectures being used for testing as a consideration in section 10.2.2.

Avoiding the potential for false confidence is crucial to successful brute-force testing. This requires careful thought over test design, not just with respect to the choice of unit for the code being tested but also with respect to the design of the test harness and the choice of testing environment. You need to ensure that as many of the code paths as possible are tested and as many of the possible thread interactions as feasible. Not only that, but you need to know *which* options are covered and *which are left untested*.

Although brute-force testing does give you some degree of confidence in your code, it's not guaranteed to find all the problems. There's one technique that *is* guaranteed to find the problems, if you have the time to apply it to your code and the appropriate software. I call it *combination simulation testing*.

Combination Simulation Testing

That's a bit of a mouthful, so I'd best explain what I mean. The idea is that you run your code with a special piece of software that *simulates* the real runtime environment of the code. You may be aware of software that allows you to run multiple virtual machines on a single physical computer, where the characteristics of the virtual machine and its hardware are emulated by the supervisor software. The idea here is similar, except rather than just emulating the system, the simulation software records the sequences of data accesses, locks, and atomic operations from each thread. It then uses the rules of the C++ memory model to repeat the run with every permitted *combination* of operations and thus identify race conditions and deadlocks.

Although such exhaustive combination testing is guaranteed to find all the problems the system is designed to detect, for anything but the most trivial